

Gray Interface '26

Task 2

1. INTRODUCTION & OBJECTIVE

In this assignment, you will move beyond data analysis and begin implementing core Machine Learning algorithms. The objective of this task is to understand how regression and classification models work, how different modeling techniques compare with one another, and how preprocessing affects model performance. You are expected to implement models, evaluate them using appropriate metrics, and provide meaningful observations rather than simply reporting scores.

2. THE ASSIGNMENT: BUILD & EVALUATE MODELS

Part 1: Linear Regression from Scratch

Before using machine learning libraries, it is important to understand how a regression model works internally.

For this part, the use of `LinearRegression` class from `sklearn` (or any equivalent pre-built implementation) is **strictly prohibited**. You are required to implement **Linear Regression using Gradient Descent** yourself in python.

Your tasks include:

- Generate a synthetic dataset of **100 samples** (you may use the following code):

```
import numpy as np
X = 2 * np.random.rand(100, 1)
noise = np.random.randn(100, 1)
y = 4 + 3 * X + noise
```

- Split the dataset into training and testing sets.
- Implement Linear Regression using Gradient Descent yourself in Python and fit it to the generated dataset.
- Visualize the generated dataset and the fitted model.
- Experiment with different learning rates and epochs.
- Plot the Loss (Cost) vs Epochs graph to visualize convergence.
- Evaluate the model using MAE, MSE, RMSE and R^2 on both training and testing datasets.
- Document your observations regarding convergence and model performance.

Bonus:

Implement **Polynomial Regression (Degree 2)** and compare its performance with your Linear Regression implementation.

Part 2: Linear Regression and Ridge/Lasso/ElasticNet Regularization

Dataset:

<https://www.kaggle.com/datasets/shashanknecrothapa/ames-housing-dataset>

Perform appropriate preprocessing before training your models. This includes data cleaning, handling missing values, encoding categorical variables, feature scaling, exploratory data analysis, and train-test splitting.

Tip: Instead of using **LinearRegression** class, use **SGDRegressor** class of sklearn as your baseline model (look up how both these classes are implemented and what benefit could this provide and mention it in the documentation)

Your tasks include:

- Train a **Linear Regression** model.
- Train **Ridge Regression**, **Lasso Regression**, and **Elastic Net** models to study and discuss the impact of regularization.
- Use cross-validation wherever applicable to determine the best regularization parameters.
- Compare all models using MAE, MSE, RMSE and R^2 on both training and testing datasets.
- Summarize which model performed best and justify your conclusion.

Part 3: Logistic Regression

Dataset:

<https://www.kaggle.com/competitions/santander-customer-transaction-prediction>

Perform appropriate preprocessing before model training, including exploratory data analysis, handling missing values (if any), feature scaling, and train-test splitting.

Your tasks include:

- Train a **Logistic Regression** model on the given dataset.
- Experiment with different values of the regularization parameter (C) and compare their impact on model performance and record them.
- Evaluate the model using Accuracy, Precision, Recall, F1-Score, ROC-AUC, Confusion Matrix
- Plot the ROC Curve and analyse the model's classification performance.
- Discuss the effect of feature scaling, regularization, and class imbalance (if observed), and summarize your findings.

3. DELIVERABLES

Submit a public link to your **Google Colab/Kaggle Notebook(s)** along with a **README.md** file. Ensure that all viewing permissions are set to public.

- **Each part of the assignment must be completed in a separate notebook file** (i.e., one notebook for Part 1, one for Part 2, and one for Part 3).
- Submit **README.md** describing your workflow, preprocessing steps, model implementation, evaluation, and the key observations and conclusions from each part,
- The documentation **does not need to be lengthy**, but it **must be written in your own words** and should clearly explain your understanding of the work performed.
- Upload all notebook files and the README.md to the GitHub repository under your own folder.

Good luck! Remember: A good Machine Learning Engineer doesn't just build models—they understand how and why they work.